Qualcomm

ELD: Rethinking link-time behavior for embedded systems at scale

Shankar Kalpathi Easwaran, Parth Arora

Qualcomm India Private Limited

partaror@qti.qualcomm.com

Agenda

The linker’s role in the Zephyr build system

Linker Overview

Why eld for Zephyr?

Key features: detailed map-file

Key features: extensive trace and warn options

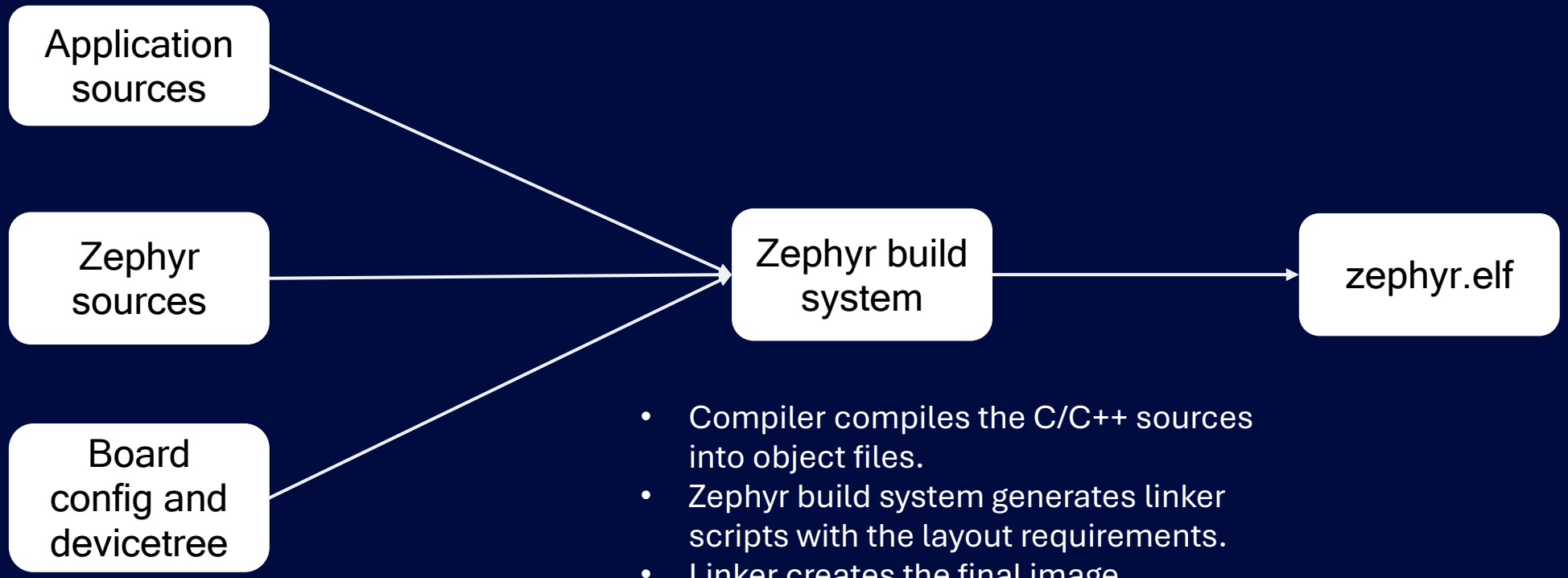
Key features: reproduce functionality

Key features: plugin framework

Future goals

students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

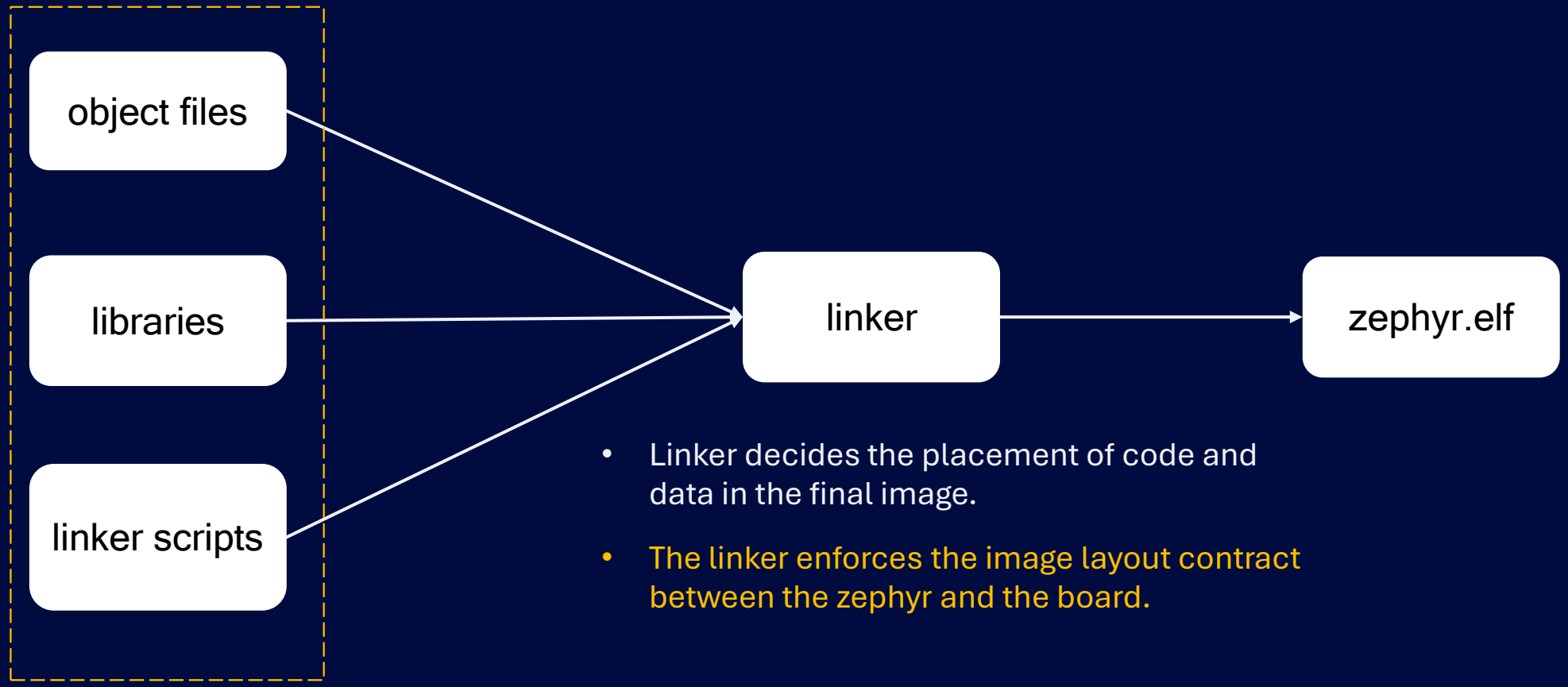
The linker's role in the zephyr build system



- Compiler compiles the C/C++ sources into object files.
- Zephyr build system generates linker scripts with the layout requirements.
- Linker creates the final image.

students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

The linker's role in the zephyr build system

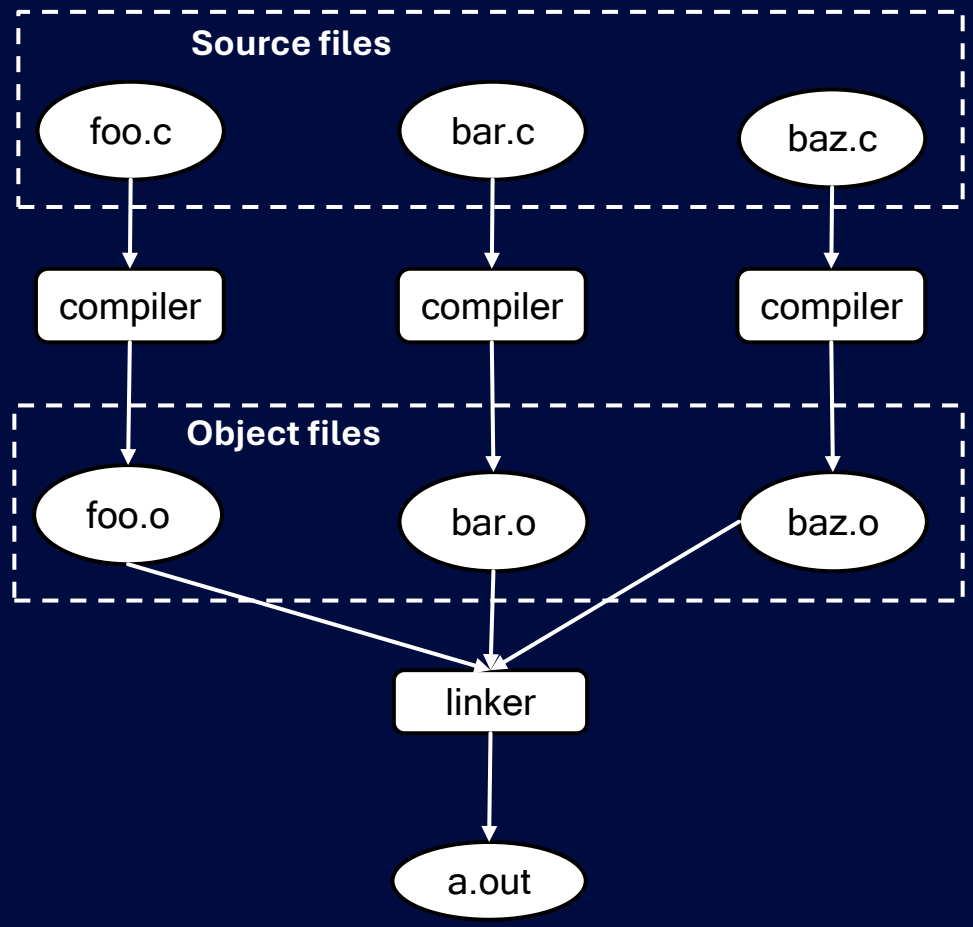


- Linker decides the placement of code and data in the final image.
- The linker enforces the image layout contract between the zephyr and the board.

students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

Linker overview

- Linker plays a key role in a toolchain
  - “Transforms all the ingredients into a meal that is ready to eat.”
- Linker operation: Read Input files
  - Resolve symbols
  - Match input sections to output sections using linker script
  - Layout image
  - Resolve relocations
  - Output image
- Widely available linkers for building applications:
  - GNU Linker, gold linker (gold), LLVM linker (lld), mclinker, Mold linker (mold), Wild linker, ELF toolchain linker



students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

Linker overview: what is so different about embedded linking?

- For embedded applications, a linker that just produces a valid ELF is not enough.
- Rich diagnostics and layout information are essential to debug issues.
- Embedded programs requires advance linker support for specifying complex image layout to match the hardware constraints and to ensure optimal performance due to section placement.
- Need of custom support in the toolchain to meet specialized requirements.
  - Overlays
  - Section budgeting
  - Security features such as embedding cryptographic signatures
- Non-embedded linking do not care about these things.

Why eld for Zephyr?

- eld is a GNU-compatible ELF linker. Designed to be used as a drop-in replacement to GNU.
- Supports ARM, AArch64, Hexagon, and RISCV.
  - x86-64 is in-progress.
- Supports GNU linker scripts for specifying complex image layout
- User friendly features for embedded development to diagnose problems and help detect undefined behaviors.
- Extensive trace, diagnostics and warning options for rapid issue identification and link-time tuning.

```
MEMORY {
  flash : ORIGIN=0x0, LENGTH = 0x1000
  ram : ORIGIN=0x1000, LENGTH = 0x2000
}

SECTIONS  {
  .text : {
    *(SORT(.text.sorted.*))
    *(.text)
  } >flash AT>flash
  .data : { *(.data) } >ram AT>ram
  …
}
```

Linker script example

students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

Why eld for Zephyr?

- Detailed map-file.
- The reproduce functionality creates standalone reproducers, crucial for replicating build issues
- Linker plugins provide precise control over image layout and enable custom functionalities for complex and specialized use-cases.
- Detailed documentation and FAQ with examples to help users understand linker behavior.
- eld is focused on providing quality and improving link behavior for Zephyr:
  - eld has nightly Zephyr build-and-test workflows: Zephyr Nightly with ELD · qualcomm/eld@bf33904
  - Enabling LLVM + eld officially in Zephyr: toolchain: llvm: add ELD (ld.eld) linker support by quic-areg · Pull Request #103778 · zephyrproject-rtos/zephyr

students to undertake on-site internships during their 7th semester for academic credit. He has expressed interest in extending his internship to cover this period.

Why eld for Zephyr: Key features

- Detailed map-file.
- Extensive trace options.
- Extensive warning options.
- The reproduce functionality for creating standalone reproducers.
- Linker plugins framework

Key features: detailed map-file

- A linker map-file describes the image layout, along with symbols and sections from individual input file.
- Map files provide a clear overview of memory regions, usage, and free space critical for embedded systems.
- Developers can analyze which object files and libraries contribute to memory sections for optimization.
- eld generates highly detailed map-file that contains linker stats, archive member information, input files information, detailed output layout, plugins information, symbol resolution details and more.
- eld map-files enables debugging of linker script expressions, symbol resolution, plugins and more.

Key features: detailed map-file

- Map file verbosity can be controlled using –MapDetail option which supports:
  - `absolute-path, show-strings, show-debug-strings`
  - `only-layout, show-initial-layout, show-header-details`
- Map-file is available in multiple formats:
  - Text format for human inspection
  - YAML format for machine parsing

Key features: detailed map-file: linker stats

- Linker stats in the map-file provides key information about the link, such as:
  - Number of object files, libraries, bitcode files, binary files, and linker scripts.
  - Number of input sections, orphan-sections, garbage-collected sections, and zero-sized sections
  - Total symbol, and discarded symbols, for example:

```
# Total Symbols: 6 { Global: 3, Local: 2, Weak: 0, Absolute: 3, Hidden: 1, Protected: 0 }
# Discarded Symbols: 1 { Global: 1, Local: 0, Weak: 0, Absolute: 0, Hidden: 1, Protected: 0 }
```

  - Number of output sections, linker script rules, and segments
  - Number of plugins
  - Number of trampolines, and relaxed (deleted) bytes.
  - Output file size
  - Link time and thread-count

Key features: detailed map-file: Output section

- ELD records resolved output section properties such as:
  - Read / write / execute attributes
  - Loadable vs non-loadable sections
  - Alignment and padding introduced during layout
  - Fill patterns used for gaps or unused memory
  - Output section virtual address, file-offset, load-memory address, alignment, and more

```
# Output Section and Layout
FOO    0x0    0xc # Offset: 0x1000, LMA: 0x0, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type: SHT_PROGBITS
*(*foo*) #Rule 1, script.t [1:0]
.text.foo       0x0     0xc     Foo.o   #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x0             foo
```

Key features: detailed map-file: linker script expression evaluation

- If a linker script assert is failing unexpectedly, then how to debug it?
- eld map-file annotates linker script expression with their values.

script.t

```
u = . + v;
assert (u == 0x1010);
```

map.txt

```
u(0x1010) = .(0x1000) + v(0x10);
assert(u(0x1010) == 0x1010);
```

Key features: extensive trace options

- The trace options provide full-visibility into linker behavior without invasive debugging techniques.

- Extensive trace options. `--trace` supports:
  - `files, symbol=<symbol-pattern>, reloc=<relocation-pattern>`
  - `command-line, linker-script, assignments, symdef`
  - `merge-strings, trampolines,`
  - `garbage-collection, live-edges,`
  - `LTO, plugin`
  - `dynamic-linking, symbol-versioning`
  - `wrap-symbols`
  - `threads`

Key features: extensive warn options

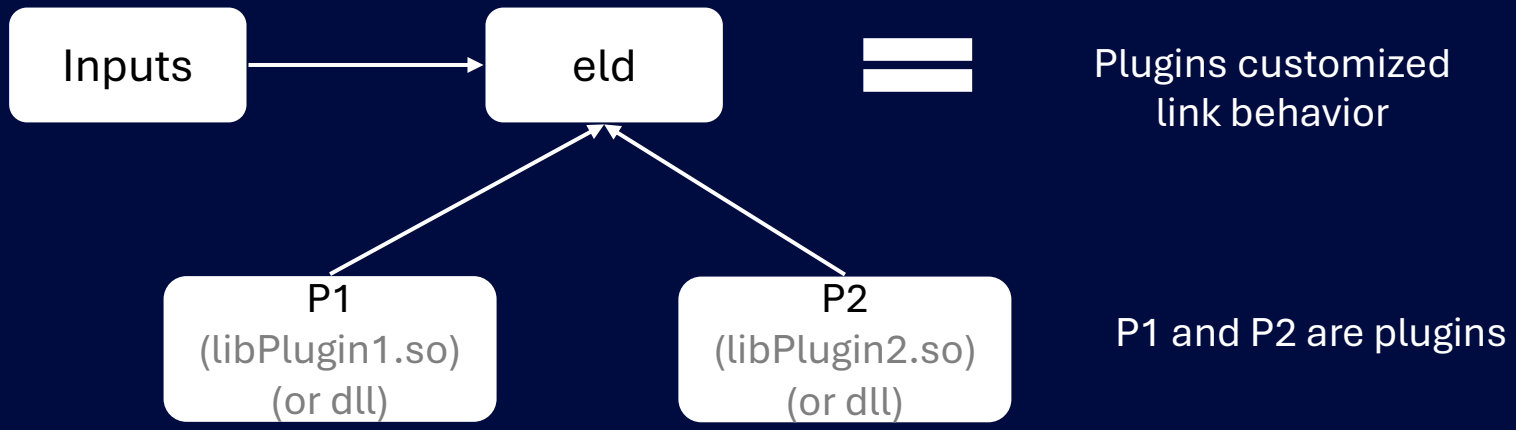
- Broad diagnostics with `-Wall`
- Fail fast enforcement with `-Werror` and `--fatal-warnings`
- Fine-grained warning control:
  - `-W[no]-linker-script`
  - `-W[no]-linker-script-memory`
  - `-W[no]-bad-dot-assignments`
  - `-W[no]-attribute-mix`
  - `-W[no]-archive-file`
  - `-W[no]-osabi`
  - `-W[no]-whole-archive`
  - `-W[no]-version-script`
  - `--[no]-warn-common`

Key features: reproduce functionality

- eld provides --reproduce functionality that creates standalone reproducer tarballs.
- Reproduce tarball bundle all link-time components into a portable artifact for exact failure reproduction.
- Reproducer tarball simplify recreating complex build environments, reducing the need to duplicate toolchains.

Key features: Plugin framework

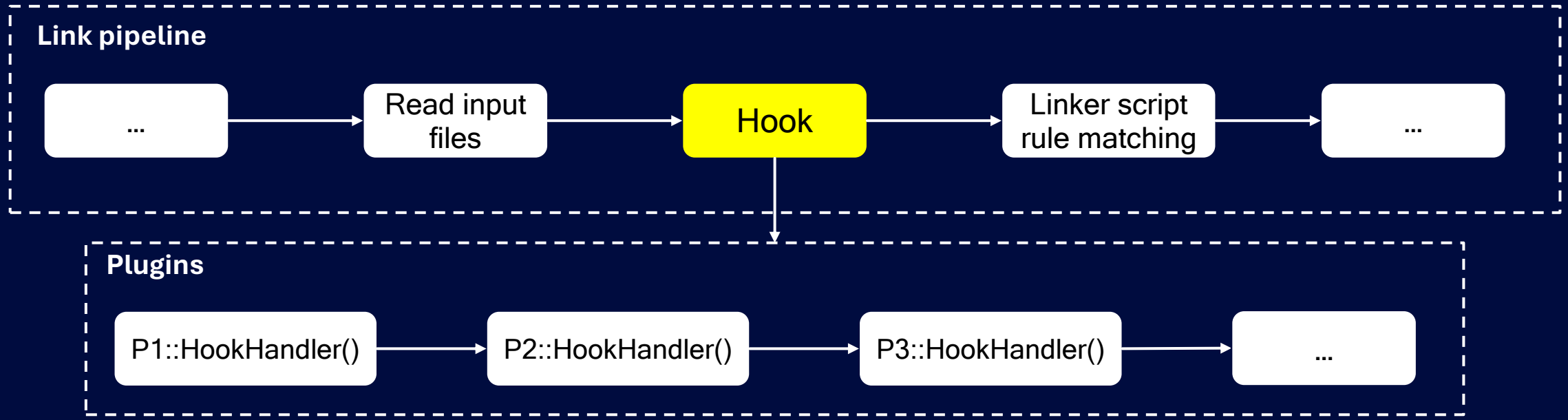
- eld plugin framework allows users to customize the link behavior without any modifications to the linker.
- The plugin framework provides finer control over the image layout than what is possible using traditional linker scripts.
- Link customization possibilities are endless with linker plugins!



Key features: Plugin framework

- Plugins can inspect and modify any of the key linker features and data structures such as: symbols, sections, relocations, input files, rule-matching, diagnostics, LTO, garbage-collection, and more.

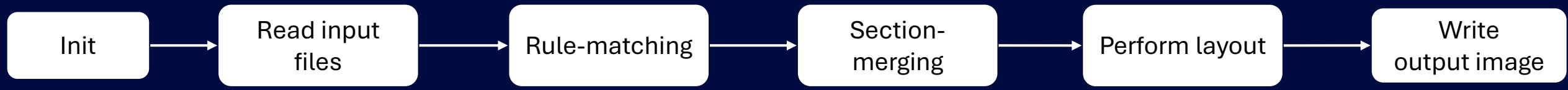
- The plugin framework provides hooks at key points of the link pipeline and plugins implement hook handlers to customize the link behavior.



Key features: Plugin framework: Plugin Hooks

- Hooks names indicate where the hook is placed within the link pipeline. There are two kinds of hooks:
  - **Visit<Component>**: These hooks are called just after creating the component. For example: VisitSection and VisitSymbol.
  - **ActBefore<LinkState>**: These hooks are called just before the linker enters a particular link state. For example: ActBeforeRuleMatching and ActBeforeSectionMerging

  - Simplified link pipeline:



Key features: Plugin Framework: use-case examples

- Plugin framework enables complex image layouts such as section placement based on PGO information and section budgeting.
- Plugin framework design can be used to create a plugin for supporting LTO with linker scripts.
- It can be used to speed-up link time by utilizing a caching layer for link steps, without any modification to the core linker functionality.
- It can be used to achieve a communication mechanism between compiler and the linker to enable specialized use-cases.

Future goals

- Feature parity with LLVM lld and GNU ld.
- Inspection tools to help debug code-size and layout issues even faster!
- LTO + linker scripts support soon with CPULLVM toolchain.
- Further performance improvements.
- Reduce memory utilization.

Thank you

Follow us on:

For more information, visit us at qualcomm.com & qualcomm.com/blog